

StockFlow

Gestion de stock et industrialisation DevOps

Architecture, implémentation et chaîne d'intégration continue

Auteur : Ghorbel Elyes

Spécialité : Ingénierie Cloud et Virtualisation

Établissement : iTeam University

Dépôt : [https://github.com/elyescesar/
pfa-gestion-stock-cicd](https://github.com/elyescesar/pfa-gestion-stock-cicd)

Table des matières

1	Introduction	7
1.1	Contexte et motivation	7
1.2	Périmètre du projet	7
1.3	Organisation du dépôt	7
1.4	Modes d'exécution	7
1.5	Organisation du rapport	8
1.6	Conventions documentées	8
2	Analyse du besoin et objectifs	9
2.1	Problématique métier	9
2.2	Acteurs et rôles	9
2.3	Exigences fonctionnelles	9
2.4	Exigences non fonctionnelles	10
2.5	Objectifs techniques	10
2.6	Hors périmètre	10
3	Architecture globale	11
3.1	Vue logique	11
3.2	Deux topologies de déploiement	11
3.2.1	Mode développement (Docker Compose)	11
3.2.2	Mode cluster (kind + Helm + Terraform)	12
3.3	Flux de données principaux	12
3.3.1	Authentification	12
3.3.2	Mouvement de stock	12
3.3.3	Observabilité	12
3.4	Chaîne d'outillage DevOps	12
3.5	Choix architecturaux structurants	12
4	Backend — API FastAPI	14
4.1	Stack et organisation du code	14
4.2	Configuration	14
4.3	Modèle de domaine	14
4.3.1	Entités	14
4.3.2	Diagramme entité-association	15
4.4	Sécurité et authentification	15
4.5	Logique métier des mouvements	15
4.6	Surface API REST	16
4.7	Export S3	16
4.8	Initialisation des données	16
4.9	Migrations Alembic	16
4.10	Conteneurisation de l'API	16
5	Frontend — interface StockFlow	17
5.1	Stack technique	17
5.2	Architecture frontale	17
5.2.1	Organisation des sources	17
5.2.2	Routage et protection	17

5.3	Client API	17
5.4	Pages fonctionnelles	18
5.4.1	Connexion (<code>PageConnexion.tsx</code>)	18
5.4.2	Tableau de bord	18
5.4.3	Produits	18
5.4.4	Catégories	18
5.4.5	Mouvements	18
5.4.6	Alertes	18
5.5	Composants UI	18
5.6	État global	18
5.7	Layout responsive	19
5.8	Build et livraison	19
5.9	Parcours utilisateur et états d'interface	19
5.10	Tests frontend	19
6	Persistance et modèle de données	20
6.1	PostgreSQL comme source de vérité	20
6.2	Schéma détaillé	20
6.2.1	Table <code>categorie</code>	20
6.2.2	Table <code>utilisateur</code>	20
6.2.3	Table <code>produit</code>	20
6.2.4	Table <code>mouvement</code>	20
6.3	Requêtes métier typiques	20
6.3.1	Alertes	20
6.3.2	Tableau de bord	21
6.4	Stratégie de démarrage du schéma	21
6.5	Données de démonstration	21
6.6	Flux transactionnel d'un mouvement	21
6.7	Relations et cardinalités	21
6.8	Index et performances	22
6.9	Comparaison SQLite (tests) et PostgreSQL (runtime)	22
6.10	Cohérence et limites	22
7	Conteneurisation Docker	23
7.1	Rôle de Docker dans le projet	23
7.2	Docker Compose — développement	23
7.2.1	Services	23
7.2.2	Variables d'environnement API	23
7.2.3	Healthchecks	23
7.2.4	Script de démarrage	23
7.3	Dockerfile backend	23
7.4	Dockerfile frontend	24
7.5	Compose outils	24
7.6	kind et chargement d'images	24
7.7	Réseau et volumes	24
7.8	Bonnes pratiques appliquées	24
8	Kubernetes et Helm	25
8.1	Kubernetes et kind	25
8.1.1	Principe	25
8.1.2	Configuration du cluster	25
8.2	Namespaces	25

8.3	Chart Helm <code>pfa-stock</code>	25
8.3.1	Métadonnées	25
8.3.2	Values principales	25
8.3.3	Ressources déployées	26
8.3.4	Ingress et routage	26
8.3.5	Sondes API	26
8.3.6	ServiceMonitor	26
8.4	ingress-nginx sur kind	26
8.5	Cycle de déploiement manuel	27
8.6	Limites du déploiement local	27
9	Infrastructure as Code — Terraform	28
9.1	Objectifs de Terraform dans StockFlow	28
9.2	Providers	28
9.2.1	AWS → LocalStack	28
9.2.2	Kubernetes et Helm	28
9.3	Variables	28
9.4	Ressources par fichier	29
9.4.1	<code>kubernetes.tf</code>	29
9.4.2	<code>localstack.tf</code>	29
9.4.3	<code>helm.tf</code>	29
9.4.4	<code>grafana_dashboard.tf</code>	29
9.5	Exécution opérationnelle	29
9.6	Validation en CI	29
9.7	Fichiers Terraform et responsabilités	29
9.8	Migration vers un cloud réel	30
10	Observabilité — Prometheus et Grafana	31
10.1	Objectifs	31
10.2	Instrumentation applicative	31
10.3	kube-prometheus-stack	31
10.4	ServiceMonitor	31
10.5	Flux de métriques	32
10.6	Dashboard Grafana	32
10.7	Accès opérationnel	32
10.8	Mode développement	32
10.9	Alerting	32
11	Intégration continue — GitHub Actions	33
11.1	Principes	33
11.2	Déclencheurs	33
11.3	Vue d'ensemble	33
11.4	Structure du fichier workflow	33
11.5	Job <code>backend</code>	34
11.6	Job <code>frontend</code>	34
11.7	Job <code>docker</code>	34
11.8	Job <code>terraform</code>	34
11.9	Job <code>helm</code>	34
11.10	Job <code>kubernetes</code>	35
11.11	Comparaison avec <code>make test</code>	35
11.12	Matrice des actions tierces	35
11.13	Échecs et gouvernance	35

11.14	Évolutions CD possibles	35
12	Sécurité, tests et qualité	36
12.1	Modèle de menace (périmètre local)	36
12.2	Authentification et autorisation	36
12.3	Secrets et configuration	36
12.4	CORS	36
12.5	Réseau Kubernetes	36
12.6	Tests automatisés	37
12.6.1	Backend (pytest)	37
12.6.2	Frontend (Vitest)	37
12.6.3	Smoke tests (<code>make verify</code>)	37
12.7	Analyse statique	37
12.8	Pistes de durcissement production	37
13	Exploitation et scripts d'automatisation	38
13.1	Makefile comme point d'entrée	38
13.2	Scripts shell	38
13.3	Prérequis machine hôte	38
13.4	Identifiants et URLs	39
13.4.1	Mode dev	39
13.4.2	Mode cluster	39
13.5	Dépannage courant	39
13.6	Logs et diagnostic	39
13.7	Sauvegarde	39
14	Conclusion et perspectives	40
14.1	Bilan	40
14.2	Objectifs atteints	40
14.3	Limites connues	40
14.4	Perspectives	40
14.5	Mot de fin	40
A	Annexe A — Référence API REST	41
A.1	Auth	41
A.2	Catégories	41
A.3	Produits	41
A.4	Mouvements	41
A.5	Tableau de bord	41
A.6	Exports	41
A.7	Santé et métriques	41
A.8	Exemples de payloads JSON	42
A.8.1	Connexion	42
A.8.2	Création de produit	42
A.8.3	Mouvement de sortie	42
A.8.4	Tableau de bord (extrait de réponse)	42
A.9	Codes d'erreur HTTP courants	42
A.10	Exemples cURL (mode Compose)	42
B	Annexe B — Configuration et versions	44
B.1	Variables d'environnement (<code>.env.example</code>)	44
B.2	Versions d'outils (référence)	44
B.3	Fichiers de référence rapide	44

Table des figures

3.1	Architecture logique de StockFlow	11
3.2	Topologie Docker Compose (développement)	11
3.3	Déploiement Kubernetes (vue simplifiée)	12
3.4	Chaîne d'outillage (build, validation, déploiement local)	13
4.1	Modèle de données simplifié	15
6.1	Séquence simplifiée — enregistrement d'un mouvement	21
8.1	Routage Ingress — chemins préservés vers l'API	26
10.1	Chemin des métriques	32
11.1	Pipeline CI — six jobs parallèles	33

Liste des tableaux

1.1	Structure logique du dépôt	8
2.1	Acteurs et permissions	9
2.2	Exigences fonctionnelles principales	9
2.3	Exigences non fonctionnelles	10
3.1	Décisions architecturales et justification	13
4.1	Tables relationnelles	14
4.2	Résumé des endpoints	16
5.1	Structure <code>frontend/src/</code>	17
5.2	Pages et endpoints consommés	19
7.1	Services Compose (développement)	23
8.1	Ressources du chart <code>pfa-stock</code>	26
9.1	Releases Helm gérées par Terraform	29
9.2	Fichiers du répertoire <code>infrastructure/terraform/</code>	30
9.3	Adaptations pour production cloud	30
11.1	CI vs commandes locales	35
11.2	Actions GitHub utilisées par job	35
12.1	Gestion des secrets	36
13.1	Commandes Make principales	38
13.2	Scripts du répertoire <code>scripts/</code>	38
A.1	Erreurs API typiques	43
B.1	Variables d'environnement principales	44

Chapitre 1

Introduction

1.1 Contexte et motivation

La gestion des stocks reste un besoin récurrent dans les PME, ateliers et entrepôts de taille modeste : suivre les quantités disponibles, enregistrer les entrées et sorties, anticiper les ruptures et disposer d'une vue consolidée pour la décision. Les tableurs partagés atteignent rapidement leurs limites (concurrence d'édition, absence d'API, traçabilité faible, déploiement non industrialisé).

StockFlow répond à ce besoin par une **application web full-stack** couplée à une **chaîne DevOps reproductible** : le même dépôt source permet de développer localement, de valider automatiquement le code (CI), de déployer sur un cluster Kubernetes local (*kind*) et de superviser l'API via Prometheus et Grafana. L'objectif architectural central est la **reproductibilité sans friction** : la machine hôte n'a besoin que de Docker, Make et Git ; Python, Node, Terraform et `kubectl` s'exécutent dans des conteneurs dédiés.

1.2 Périmètre du projet

Le périmètre fonctionnel couvre :

- authentification par jeton JWT avec rôles **admin** et **opérateur** ;
- gestion des catégories et des produits (SKU, quantité, seuil d'alerte) ;
- mouvements de stock (**entree** / **sortie**) avec contrôle de stock négatif ;
- tableau de bord synthétique et liste des produits sous le seuil ;
- export CSV des produits vers un bucket S3 (LocalStack en environnement local).

Le périmètre technique couvre :

- API REST FastAPI et interface React (Vite, TypeScript, Tailwind) ;
- persistance PostgreSQL 16 ;
- conteneurisation Docker et orchestration Docker Compose pour le développement ;
- déploiement sur cluster **kind** via chart Helm **pfa-stock** ;
- provisionnement Terraform (S3 LocalStack, releases Helm) ;
- stack **kube-prometheus-stack** avec dashboard Grafana versionné ;
- pipeline GitHub Actions à six jobs parallèles.

1.3 Organisation du dépôt

Le dépôt suit une séparation nette des responsabilités (tableau [1.1](#)).

1.4 Modes d'exécution

Deux modes coexistent et répondent à des objectifs différents :

make dev Stack Docker Compose : frontend sur le port 3000, API sur le port 8000. Boucle de développement rapide, CORS entre origines distinctes.

make cluster Cluster kind + Terraform + Helm : application accessible sur `http://localhost` (port 80), routage Ingress unique, monitoring complet.

TABLE 1.1 – Structure logique du dépôt

Répertoire	Rôle
backend/	API FastAPI, modèles SQLAlchemy, Alembic, tests py-test
frontend/	SPA React, build Vite, tests Vitest
docker/	Fichiers Compose (dev et outils)
helm/pfa-stock/	Chart Kubernetes de l'application
infrastructure/	Configuration kind et code Terraform
monitoring/	Valeurs Prometheus/Grafana et dashboard JSON
scripts/	Automatisation démarrage, vérification, réparation
.github/workflows/	Pipeline CI

1.5 Organisation du rapport

Le chapitre 2 formalise les objectifs et les acteurs. Le chapitre 3 présente l'architecture globale et les flux principaux. Les chapitres 4 à 6 détaillent l'application ; les chapitres 7 à 9 l'infrastructure ; les chapitres 10 et 11 l'observabilité et l'intégration continue. Le chapitre 13 documente l'exploitation quotidienne. Les annexes recensent l'API REST et les variables de configuration.

1.6 Conventions documentées

- Les chemins de fichiers sont relatifs à la racine du dépôt sauf mention contraire.
- Les identifiants de démonstration (`admin@stock.tn`, etc.) sont documentés à titre opérationnel ; en production ils doivent être remplacés et stockés dans un gestionnaire de secrets.
- Les captures d'écran de l'interface ne sont pas reproduites ici : le lecteur peut les générer via `make demo` ou `make demo-cluster`.

Chapitre 2

Analyse du besoin et objectifs

2.1 Problématique métier

Un gestionnaire de stock doit pouvoir répondre rapidement aux questions suivantes :

1. Quels produits sont en rupture ou proches du seuil minimum ?
2. Quelles entrées et sorties ont eu lieu, par qui, et pour quel motif ?
3. Comment organiser le catalogue par familles (catégories) ?
4. Comment exporter l'état du stock pour archivage ou traitement externe ?

Ces besoins imposent une base relationnelle cohérente, des règles métier transactionnelles (un mouvement de sortie ne doit pas rendre le stock négatif) et une interface qui reflète l'état en temps quasi réel après chaque opération.

2.2 Acteurs et rôles

TABLE 2.1 – Acteurs et permissions

Rôle	Compte démo	Capacités
Administrateur	<code>admin@stock.tn</code>	CRUD complet, suppression catégories/produits, export CSV S3
Opérateur	<code>opérateur@stock.tn</code>	Consultation, création mouvements et entités (sauf suppressions réservées admin)

L'authentification repose sur un jeton JWT transmis dans l'en-tête **Authorization: Bearer**.
Le backend résout l'utilisateur courant à chaque requête protégée via la dépendance `obtenir_utilisateur_com`

2.3 Exigences fonctionnelles

TABLE 2.2 – Exigences fonctionnelles principales

ID	Description
F1	Connexion par email et mot de passe, émission d'un jeton d'accès.
F2	CRUD catégories (nom, description optionnelle).
F3	CRUD produits (nom, référence SKU unique, quantité, seuil d'alerte, catégorie).
F4	Enregistrement de mouvements entree ou sortie avec mise à jour atomique du stock.
F5	Rejet d'une sortie si le stock deviendrait négatif (stock_insuffisant).
F6	Tableau de bord : compteurs produits, catégories, mouvements, liste des alertes.
F7	Page alertes : produits dont quantite $\leq$ seuil_alerte .
F8	Export CSV de tous les produits vers S3 (réservé admin).

2.4 Exigences non fonctionnelles

TABLE 2.3 – Exigences non fonctionnelles

ID	Description
NF1	Conteneurisation : exécution reproductible via Docker.
NF2	Orchestration : déploiement sur Kubernetes avec Ingress, sondes de santé, ServiceMonitor.
NF3	IaC : infrastructure déclarative (Terraform + Helm).
NF4	Observabilité : métriques Prometheus sur <code>/metrics</code> , dashboard Grafana.
NF5	CI : validation automatique (lint, tests, build images, validate Terraform/Helm).
NF6	Sécurité : mots de passe hashés (bcrypt), secrets Kubernetes pour données sensibles.
NF7	Documentation opérationnelle : README, Makefile, scripts <code>make demo</code> .

2.5 Objectifs techniques

Les objectifs techniques déclinés dans l'implémentation sont :

1. **Séparation des couches** : frontend statique, API stateless, base PostgreSQL, stockage objet pour les exports.
2. **Deux chemins de déploiement** partageant les mêmes Dockerfiles : Compose (vélocité) et kind/Helm (réalisme production).
3. **Automatisation bout en bout** : un développeur clone le dépôt, installe Docker, exécute `make demo-cluster` et obtient l'application, le monitoring et le bucket S3 sans configuration manuelle dispersée.
4. **Qualité logicielle mesurable** : tests unitaires et d'intégration API, lint (ruff, eslint), smoke tests shell (`make verify`).
5. **Préparation cloud** : providers Terraform et chart Helm conçus pour une migration vers AWS/EKS en changeant endpoints et backends, sans réécriture applicative.

2.6 Hors périmètre

Les éléments suivants sont explicitement hors périmètre de la version documentée :

- déploiement continu automatique vers un environnement distant (pas de `helm upgrade` en CI) ;
- haute disponibilité multi-nœuds, autoscaling horizontal, TLS terminé par certificats réels ;
- gestion fine des permissions au-delà de deux rôles ;
- application mobile native ou mode hors-ligne.

Chapitre 3

Architecture globale

3.1 Vue logique

L'architecture suit un modèle en couches classique, enrichi d'une couche d'observabilité et d'un stockage objet pour les exports. La figure 3.1 synthétise les composants et leurs interactions.

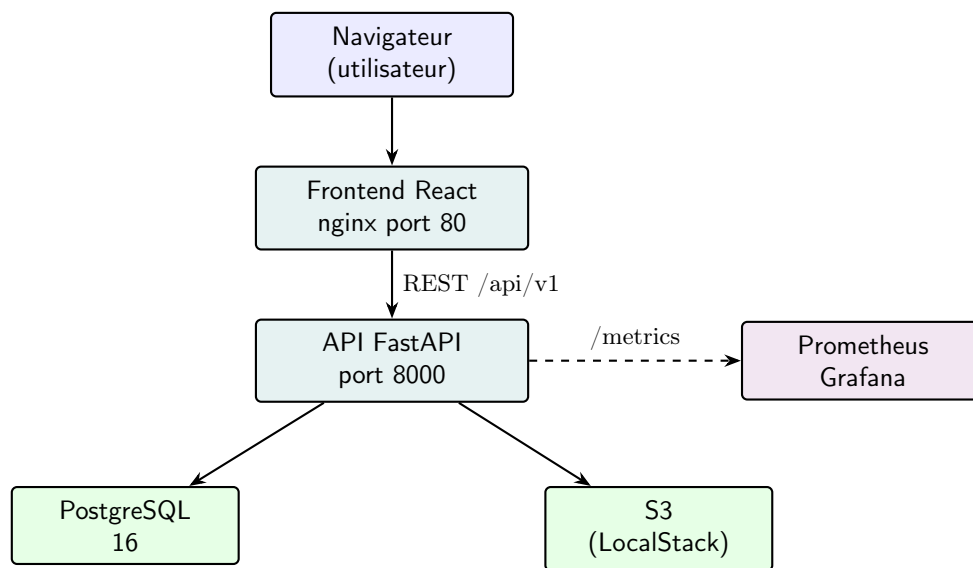


FIGURE 3.1 – Architecture logique de StockFlow

3.2 Deux topologies de déploiement

3.2.1 Mode développement (Docker Compose)

Le fichier `docker/compose.dev.yml` définit un réseau bridge `pfa` reliant quatre services. La figure 3.2 illustre les dépendances et les ports publiés sur l'hôte.

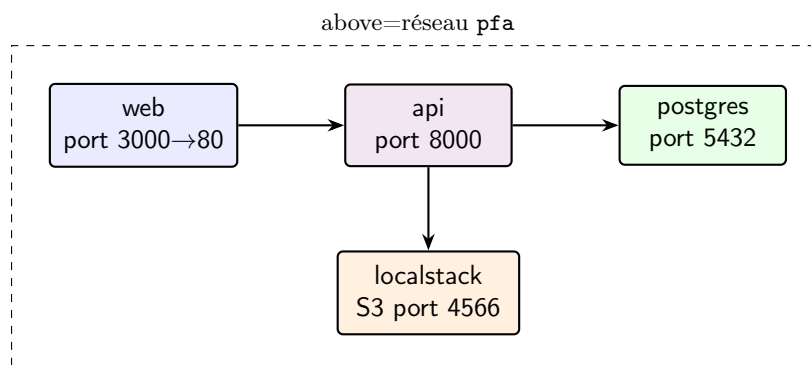


FIGURE 3.2 – Topologie Docker Compose (développement)

Les *healthchecks* ordonnent le démarrage : PostgreSQL doit répondre à `pg_isready` avant que l'API ne démarre ; l'API doit répondre sur `/health` avant que le conteneur web ne soit considéré prêt.

3.2.2 Mode cluster (kind + Helm + Terraform)

En mode cluster, un seul point d'entrée HTTP (port 80) est exposé via **ingress-nginx**. L'Ingress route le préfixe `/api` vers le Service `api` et le reste vers `web`. La figure 3.3 représente les namespaces et les objets principaux.

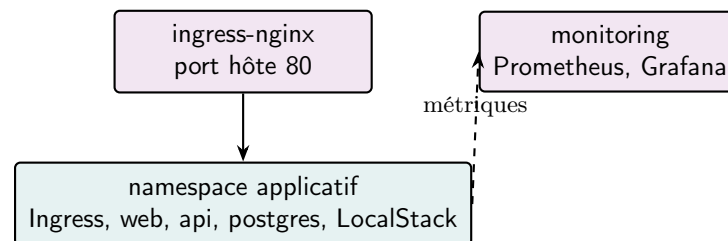


FIGURE 3.3 – Déploiement Kubernetes (vue simplifiée)

Terraform crée les namespaces, le bucket S3 sur LocalStack, puis installe séquentiellement `ingress-nginx`, `kube-prometheus-stack` et le chart `pfa-stock` (dépendances `depends_on` dans `helm.tf`).

3.3 Flux de données principaux

3.3.1 Authentification

1. Le client envoie `POST /api/v1/auth/connexion` avec email et mot de passe.
2. L'API vérifie le hash `bcrypt`, émet un JWT (claims `sub`, `role`, expiration configurable).
3. Le frontend stocke le jeton dans `sessionStorage` et l'envoie sur chaque requête.
4. `GET /api/v1/auth/moi` permet de recharger le profil au chargement de l'application.

3.3.2 Mouvement de stock

Le service `gestion_stock.py` encapsule la logique transactionnelle : lecture du produit, calcul de la nouvelle quantité, rejet si négatif, insertion du mouvement et commit. Ce flux est détaillé au chapitre 4.

3.3.3 Observabilité

L'instrumentateur Prometheus enregistre chaque requête HTTP. En cluster, le `ServiceMonitor` du chart expose le port `http` et le chemin `/metrics` au Prometheus Operator (chapitre 10).

3.4 Chaîne d'outillage DevOps

La figure 3.4 situe les outils par rapport au cycle de vie du code.

3.5 Choix architecturaux structurants

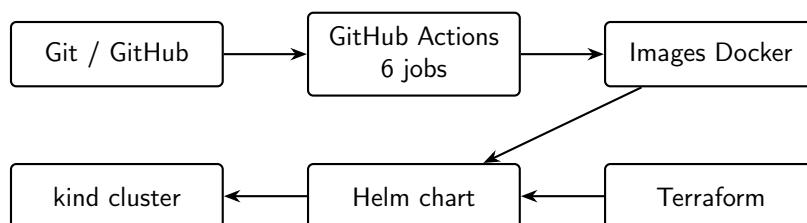


FIGURE 3.4 – Chaîne d’outillage (build, validation, déploiement local)

TABLE 3.1 – Décisions architecturales et justification

Décision	Justification
API stateless	Facilite le scaling horizontal des pods API ; session portée par JWT.
Frontend statique (nginx)	Réduction de la surface d’attaque et du coût runtime en cluster.
PostgreSQL relationnel	Intégrité référentielle, transactions ACID pour les mouvements.
kind + Terraform + Helm	Reproductibilité d’un environnement proche production sans cloud.
LocalStack S3	Démonstration IaC cloud sans compte AWS.
create_all + Alembic	Démarrage simple en dev ; migrations versionnées pour traçabilité du schéma.

Chapitre 4

Backend — API FastAPI

4.1 Stack et organisation du code

Le backend est implémenté en **Python 3.12** avec **FastAPI 0.15**. Les dépendances principales figurent dans `backend/requirements.txt` : SQLAlchemy 2, Alembic, Pydantic Settings, `python-jose`, `passlib/bcrypt`, `prometheus-fastapi-instrumentator`, `boto3`.

L'arborescence `backend/app/` suit une séparation routes / schémas / modèles / services :

- `modeles/` — entités SQLAlchemy ;
- `schemas/` — modèles Pydantic entrée/sortie ;
- `routes/` — routeurs FastAPI par domaine ;
- `securite/` — JWT et hachage des mots de passe ;
- `services/` — logique métier (stock, export S3).

Le point d'entrée `main.py` instancie l'application, configure CORS, monte les routeurs sous le préfixe `/api/v1`, expose `/health` et instrumente `/metrics`.

4.2 Configuration

La classe `Parametres` (`config.py`) charge les variables d'environnement via `pydantic-settings` : `DATABASE_URL`, `SECRET_JWT`, `CORS_ORIGINS`, paramètres AWS/LocalStack, durée du jeton JWT, algorithme HS256.

En mode test (`MODE_TEST=1`), le cycle de vie de l'application ne crée pas les tables ni n'exécute le seed global : les tests utilisent une base SQLite en mémoire avec fixtures dédiées (`tests/conftest.py`).

4.3 Modèle de domaine

4.3.1 Entités

TABLE 4.1 – Tables relationnelles

Table	Clés	Attributs notables
<code>categorie</code>	PK id	nom unique, description nullable
<code>utilisateur</code>	PK id	email unique, mot_de_passe_hash, role
<code>produit</code>	PK, FK id_categorie	reference_sku unique, quantite, seuil_alerte
<code>mouvement</code>	PK, FK produit + utilisateur	type_mouvement, quantite, motif, date_mouvement

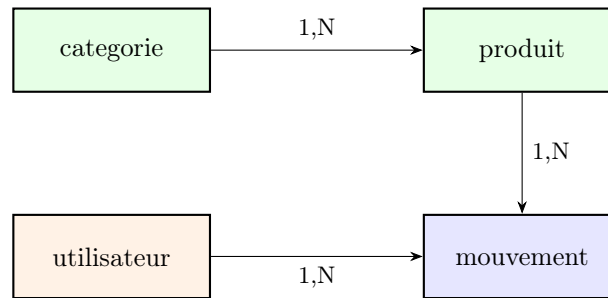


FIGURE 4.1 – Modèle de données simplifié

4.3.2 Diagramme entité-association

4.4 Sécurité et authentification

Les mots de passe ne sont jamais stockés en clair : `mots_de_passe.py` utilise `bcrypt`. Le module `JWT` (`securite/jwt.py`) :

Listing 4.1 – Émission et validation JWT (extrait)

```

1 def creer_jeton_acces(donnees: dict) -> str:
2     expiration = datetime.now(timezone.utc) + timedelta(
3         minutes=parametres.duree_token_minutes)
4     charge = donnees.copy()
5     charge.update({"exp": expiration})
6     return jwt.encode(charge, parametres.secret_jwt,
7                       algorithm=parametres.algorithme_jwt)

```

La dépendance `obtenir_utilisateur_courant` decode le jeton, extrait `sub` (email), charge l'utilisateur en base ou renvoie 401 `jeton_invalide`.

4.5 Logique métier des mouvements

Le cœur métier réside dans `services/gestion_stock.py`. L'algorithme :

1. Charger le produit par `id_produit` ; 404 si absent.
2. Calculer `nouvelle_quantite` : addition pour `entree`, soustraction pour `sortie`.
3. Si `nouvelle_quantite < 0`, lever HTTP 400 avec `detail=stock_insuffisant`.
4. Créer l'enregistrement `Mouvement`, mettre à jour `produit.quantite`, `commit`.

Listing 4.2 – Création d'un mouvement (extrait)

```

1 def creer_mouvement(session, donnees, id_utilisateur):
2     produit = session.query(Produit).filter(
3         Produit.id == donnees.id_produit).first()
4     if produit is None:
5         raise HTTPException(status_code=404, detail="
6             produit_introuvable")
7     nouvelle_quantite = calculer_quantite_apres_mouvement(
8         produit.quantite, donnees.type_mouvement, donnees.quantite)
9     if nouvelle_quantite < 0:
10        raise HTTPException(status_code=400, detail="stock_insuffisant
11            ")
12    # ... commit mouvement + mise      jour produit

```


4.6 Surface API REST

Tous les endpoints métier sont préfixés par `/api/v1`. Le tableau 4.2 résume les routes ; l'annexe A détaille les payloads.

TABLE 4.2 – Résumé des endpoints

Méthode	Chemins	Auth	Rôle
POST	<code>/auth/connexion</code>	Non	Émet JWT
GET	<code>/auth/moi</code>	Oui	Profil
CRUD	<code>/categories</code>	Oui	DELETE admin
CRUD	<code>/produits</code>	Oui	DELETE admin
GET	<code>/produits/alertes</code>	Oui	Sous seuil
GET/POST	<code>/mouvements</code>	Oui	Historique / création
GET	<code>/tableau-de-bord</code>	Oui	Agrégats
POST	<code>/exports/produits-csv</code>	Admin	Export S3
GET	<code>/health, /metrics</code>	Non	Sonde / Prometheus

4.7 Export S3

`services/export_s3.py` génère un CSV des produits en mémoire et l'envoi via boto3 vers le bucket configuré (`pfa-stock-exports`). Le client boto3 utilise `AWS_ENDPOINT_URL` pour cibler LocalStack en local. L'endpoint POST `/exports/produits-csv` vérifie le rôle administrateur avant exécution.

4.8 Initialisation des données

`seed.py` peuple la base au premier démarrage (hors tests) : utilisateurs admin et opérateur, catégories et produits de démonstration. Cela permet une expérience immédiate après `make dev` sans script SQL manuel.

4.9 Migrations Alembic

Le répertoire `alembic/versions/` contient la migration initiale `001_initial.py` qui crée les quatre tables. `alembic/env.py` importe tous les modèles pour alimenter `target_metadata`. En exploitation courante, `Base.metadata.create_all` au démarrage assure la présence du schéma ; Alembic reste la référence versionnée pour audits et évolutions futures.

4.10 Conteneurisation de l'API

Le `backend/Dockerfile` utilise `python:3.12-slim`, installe `libpq-dev` pour `psycopg2`, copie `requirements.txt` puis le code, expose le port 8000 et lance :

```
1 uvicorn app.main:application --host 0.0.0.0 --port 8000
```

Les sondes Kubernetes (startup et liveness) ciblent GET `/health` sur le port 8000 (définies dans le template Helm `api.yaml`).

Chapitre 5

Frontend — interface StockFlow

5.1 Stack technique

L'interface utilisateur est une **single-page application** construite avec :

- **React 18** et **TypeScript 5.6**;
- **Vite 6** comme bundler et serveur de développement ;
- **React Router 7** pour le routage côté client ;
- **Tailwind CSS 4** via le plugin `@tailwindcss/vite` ;
- **lucide-react** pour l'iconographie ;
- **Vitest** et **React Testing Library** pour les tests.

Le nom produit affiché est **StockFlow** : thème sombre, accents cyan, typographie Outfit (chargée depuis Google Fonts dans `index.html`).

5.2 Architecture frontale

5.2.1 Organisation des sources

TABLE 5.1 – Structure `frontend/src/`

Dossier	Contenu
<code>pages/</code>	Une page par route métier
<code>composants/Layout.tsx</code>	Shell navigation + zone <code>Outlet</code>
<code>composants/ui/</code>	Bibliothèque de composants réutilisables
<code>contexte/</code>	Auth et toasts globaux
<code>hooks/useFetch.ts</code>	Chargement données API avec état
<code>services/api.ts</code>	Client HTTP, gestion du jeton JWT

5.2.2 Routage et protection

`App.tsx` définit :

- `/connexion` — publique ;
- routes imbriquées sous `Layout` — protégées par `RoutesProtegees` qui lit `AuthContext` ;
- redirection vers `/connexion` si non authentifié ;
- affichage d'un chargeur pendant la vérification du jeton (`GET /auth/moi`).

5.3 Client API

`services/api.ts` centralise les appels :

- base URL depuis `import.meta.env.VITE_API_URL` (injectée au build Docker) ;
- jeton stocké dans `sessionStorage` sous la clé `jeton_acces` ;
- fonction `appelApi<T>()` ajoutant l'en-tête `Authorization` et gérant les erreurs 401 (déconnexion implicite).

En mode cluster, le frontend est servi depuis `http://localhost` et les appels partent vers `http://localhost/api/...` via l’Ingress — d’où le build avec `VITE_API_URL=http://localhost`.

5.4 Pages fonctionnelles

5.4.1 Connexion (`PageConnexion.tsx`)

Formulaire email / mot de passe, valeurs préremplies en démo. Appel `POST /auth/connexion`, stockage du jeton, toast de succès, navigation vers le tableau de bord. Redirection automatique si session déjà valide.

5.4.2 Tableau de bord

Consomme `GET /api/v1/tableau-de-bord`. Affiche quatre cartes statistiques (produits, catégories, mouvements, alertes) et une liste des produits sous seuil avec barres de progression visuelles.

5.4.3 Produits

Liste filtrable (nom, SKU, catégorie), tableau avec badge d’alerte si stock bas, modal de création (`POST`), suppression avec confirmation (`DELETE`, visible au survol).

5.4.4 Catégories

Grille de cartes avec description tronquée, modal de création, suppression au survol.

5.4.5 Mouvements

Timeline verticale : entrées (vert) et sorties (rouge), date et motif. Modal pour enregistrer un mouvement (type, produit, quantité, motif optionnel).

5.4.6 Alertes

Cartes par produit en alerte avec jauge et pourcentage par rapport au seuil ; lien vers les mouvements pour corriger le stock.

5.5 Composants UI

La couche `composants/ui/` homogénéise l’interface :

- `Bouton` — variantes primaire, secondaire, danger, fantôme ; état chargement ;
- `Modal` — fermeture `Escape`, tailles `md/lg` ;
- `Champ`, `Input`, `Select`, `Textarea` ;
- `CarteStat`, `Badge`, `Chargeur`, `EtatVide`, `BarreRecherche`.

5.6 État global

`AuthContext` expose `utilisateur`, `chargement`, `connexion`, `deconnexion`. Au montage, si un jeton existe en session, appel `/auth/moi` pour hydrater le profil.

`ToastContext` permet d’afficher des notifications succès / erreur / info en bas à droite (auto-dismiss après 4 secondes).

5.7 Layout responsive

`Layout.tsx` combine :

- barre latérale fixe sur grand écran (navigation, profil, déconnexion) ;
- menu hamburger et tiroir sur mobile ;
- animation `animate-fade-in` à chaque changement de route (`key={location.pathname}`).

5.8 Build et livraison

Le `frontend/Dockerfile` est multi-stage :

1. Stage build : `node:20-alpine`, `npm install`, `npm run build` (TypeScript + Vite).
2. Stage runtime : `nginx:alpine`, copie de `dist/` et `nginx.conf`.

`nginx.conf` configure le fallback SPA (`try_files`) et, en Compose, un proxy `/api/` vers le service `api` — en cluster, c'est l'Ingress qui assume ce routage.

5.9 Parcours utilisateur et états d'interface

Chaque page métier suit le même patron : chargement (`Chargeur`), état vide (`EtatVide`) si la liste est vide, puis contenu principal. Les actions destructives (suppression) utilisent une confirmation implicite via bouton dédié et toast d'erreur en cas d'échec API.

TABLE 5.2 – Pages et endpoints consommés

Route	Endpoints principaux
<code>/connexion</code>	POST <code>/auth/connexion</code>
<code>/</code> (dashboard)	GET <code>/tableau-de-bord</code>
<code>/produits</code>	GET/POST/DELETE <code>/produits</code>
<code>/categories</code>	GET/POST/DELETE <code>/categories</code>
<code>/mouvements</code>	GET/POST <code>/mouvements</code>
<code>/alertes</code>	GET <code>/produits/alertes</code>

Le hook `useFetch` encapsule le cycle *chargement* → *données* → *erreur*, ce qui évite de dupliquer les `useEffect` sur chaque page.

5.10 Tests frontend

`App.test.tsx` vérifie le rendu du titre « Connexion » avec les fournisseurs de contexte et un `MemoryRouter`. La CI exécute `npm run lint` (ESLint) et `npm test - --run` (Vitest).

Les règles ESLint (config flat dans `eslint.config.js`) couvrent les hooks React et les imports inutilisés ; elles complètent TypeScript qui reste la première barrière de typage au build `tsc -b`.

Chapitre 6

Persistance et modèle de données

6.1 PostgreSQL comme source de vérité

Toutes les données métier résident dans **PostgreSQL 16**. Le moteur est choisi pour :

- les transactions ACID (mouvement + mise à jour quantité en un seul commit) ;
- les contraintes d'intégrité (unicité SKU, clés étrangères) ;
- la compatibilité entre dev (Compose), CI (service container, tests SQLite séparés) et cluster (StatefulSet).

6.2 Schéma détaillé

6.2.1 Table categorie

Listing 6.1 – Structure logique categorie

```
1 CREATE TABLE categorie (  
2   id SERIAL PRIMARY KEY,  
3   nom VARCHAR UNIQUE NOT NULL,  
4   description TEXT  
5 );
```

6.2.2 Table utilisateur

Stocke `mot_de_passe_hash` uniquement. Le champ `role` accepte `admin` ou `operateur` (contrôlé applicativement).

6.2.3 Table produit

- `reference_sku` — identifiant métier unique ;
- `quantite` — stock courant, entier ≥ 0 après mouvements valides ;
- `seuil_alerte` — déclenche l'alerte lorsque `quantite` $\leq$ `seuil_alerte`.

6.2.4 Table mouvement

Journal append-only des opérations : `type_mouvement` $\in$ {entree, sortie}, horodatage `date_mouvement`, lien vers l'auteur (`id_utilisateur`).

6.3 Requêtes métier typiques

6.3.1 Alertes

GET /produits/alertes exécute un filtre SQL du type :

```
1 SELECT * FROM produit WHERE quantite <= seuil_alerte ORDER BY quantite  
   ;
```

6.3.2 Tableau de bord

Agrégations : `COUNT` sur produits, catégories, mouvements ; sous-requête ou filtre pour les produits en alerte renvoyés dans le même payload JSON.

6.4 Stratégie de démarrage du schéma

Deux mécanismes coexistent :

1. **Runtime** : `create_all` dans le lifespan FastAPI crée les tables si absentes, puis `seed.py`.
2. **Alembic** : migration `001_initial` versionnée pour documentation et pipelines futurs (`alembic upgrade head`).

En cluster, le StatefulSet PostgreSQL monte un volume persistant (`postgres.storage: 1Gi` dans `values.yaml`) : les données survivent au redémarrage du pod sous réserve du PVC.

6.5 Données de démonstration

Le seed crée notamment :

- `admin@stock.tn / Admin123!` ;
- `opérateur@stock.tn / Operateur123!` ;
- catégories et produits avec quantités variées pour illustrer les alertes.

6.6 Flux transactionnel d'un mouvement

La figure 6.1 décrit les échanges lors d'une sortie de stock : une seule transaction SQLAlchemy regroupe l'insertion du mouvement et la mise à jour de `produit.quantite`. En cas d'erreur avant le `commit`, aucune des deux écritures n'est persistée.

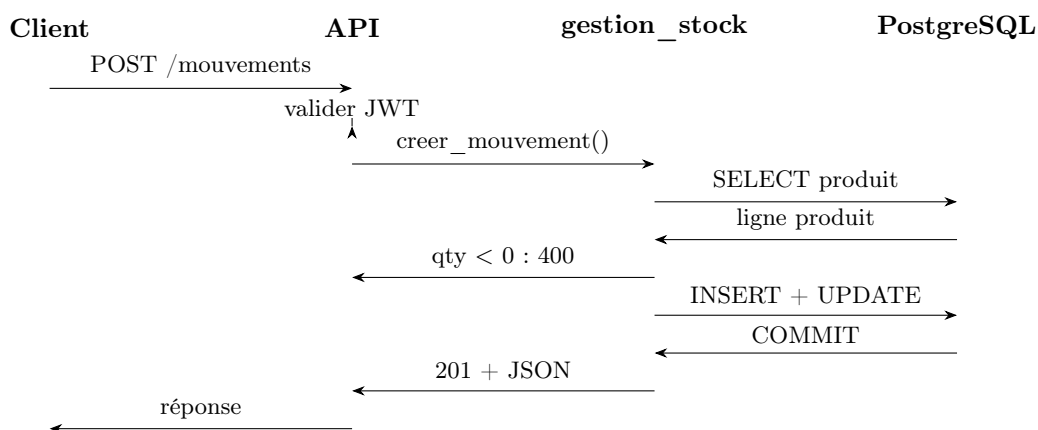


FIGURE 6.1 – Séquence simplifiée — enregistrement d'un mouvement

6.7 Relations et cardinalités

- Une **catégorie** regroupe plusieurs **produits** ; la suppression d'une catégorie référencée est bloquée par la contrainte FK (comportement REST : erreur applicative ou 409 selon le cas).
- Un **produit** possède un historique de **mouvements** ordonné par `date_mouvement` décroissant côté API.
- Chaque mouvement enregistre l'**utilisateur** auteur via `id_utilisateur`, ce qui permet une traçabilité minimale (qui a saisi l'entrée ou la sortie).

6.8 Index et performances

Le schéma initial ne déclare pas d'index secondaires explicites au-delà des clés primaires et des contraintes `UNIQUE` sur `nom` (catégorie), `email` et `reference_sku`. Pour un volume de démonstration (quelques dizaines de produits), les requêtes restent instantanées. Une montée en charge imposerait typiquement :

- un index sur `produit(id_categorie)` pour les jointures catalogue ;
- un index composite (`quantite`, `seuil_alerte`) ou une vue matérialisée pour les alertes si le filtre devient coûteux ;
- un index sur `mouvement(id_produit, date_mouvement DESC)` pour l'historique paginé.

6.9 Comparaison SQLite (tests) et PostgreSQL (runtime)

Les tests unitaires (`backend/tests/`) utilisent SQLite en mémoire : le dialecte SQLAlchemy reste identique pour les opérations CRUD simples, mais certaines subtilités (types, contraintes) diffèrent. Le runtime Compose et cluster cible PostgreSQL 16 ; c'est la référence pour les types `SERIAL`, `TEXT` et le comportement transactionnel réel.

6.10 Cohérence et limites

- Pas de verrouillage pessimiste explicite : en concurrence élevée, deux sorties simultanées pourraient nécessiter `SELECT FOR UPDATE` ; le périmètre actuel suppose une charge modérée.
- Pas de soft-delete : les `DELETE` sont physiques (réservés admin).
- Pas de réplication PostgreSQL en local : single instance.
- Les exports CSV ne modifient pas le schéma : ils lisent un snapshot des produits au moment de l'appel admin.

Chapitre 7

Conteneurisation Docker

7.1 Rôle de Docker dans le projet

Docker matérialise l’environnement d’exécution : mêmes versions de runtime, mêmes dépendances système, isolation des services. Deux fichiers Compose distincts évitent de mélanger le quotidien développeur et les outils d’infrastructure lourds.

7.2 Docker Compose — développement

7.2.1 Services

Le tableau 7.1 détaille `docker/compose.dev.yml`.

TABLE 7.1 – Services Compose (développement)

Service	Image	Port hôte	Rôle
postgres	postgres :16-alpine	5432	Base <code>stock_db</code>
api	build backend	8000	FastAPI
web	build frontend	3000→80	nginx + SPA
localstack	localstack :3	4566	API S3

7.2.2 Variables d’environnement API

L’API reçoit notamment :

- `DATABASE_URL=postgresql://stock:stock@postgres:5432/stock_db`;
- `SECRET_JWT`, `CORS_ORIGINS` incluant `http://localhost:3000`;
- `AWS_ENDPOINT_URL=http://localstack:4566`, bucket `pfa-stock-exports`.

7.2.3 Healthchecks

- PostgreSQL : `pg_isready` toutes les 5 s;
- API : requête Python vers `http://localhost:8000/health`;
- `depends_on` avec condition `service_healthy` enchaîne `postgres` → `api` → `web`.

7.2.4 Script de démarrage

`scripts/demarrer-dev.sh` copie `.env.example` vers `.env` si nécessaire, lance `compose up -build`, exécute `init-localstack.sh` pour créer le bucket S3.

7.3 Dockerfile backend

Image basée sur `python:3.12-slim` :

- installation de `libpq-dev` et `gcc` pour compiler `psycopg2`;
- couche de dépendances `pip` mise en cache si `requirements.txt` inchangé;

- `PYTHONPATH=/app` pour résoudre le package `app`.

7.4 Dockerfile frontend

Build multi-stage (chapitre 5) : l'argument `VITE_API_URL` est figé au build. En dev Compose, `http://localhost:8000` ; en cluster, `http://localhost` pour que le navigateur appelle l'Ingress.

7.5 Compose outils

`docker/compose.outils.yml` sert au bootstrap cluster :

- **localstack** — endpoint S3 pour Terraform ;
- **terraform** — image `hashicorp/terraform:1.9`, volumes montés sur le repo et `.kube`, réseau host ;
- **kubectl** / **helm** — CLI dans des conteneurs éphémères.

Aucune installation de Terraform ou kubectl sur l'hôte n'est requise pour `make cluster`.

7.6 kind et chargement d'images

Le script `demarrer-cluster.sh` :

1. build `pfa-stock-api:latest` et `pfa-stock-web:latest` ;
2. crée le cluster kind avec `kind-docker.sh` (kind exécuté dans un conteneur avec accès au socket Docker) ;
3. exporte le kubeconfig vers `.kube/config` ;
4. `kind load docker-image` injecte les images locales dans le cluster (évite un registry).

7.7 Réseau et volumes

- Réseau `pfa` (bridge) — résolution DNS par nom de service ;
- Volume nommé `donnees_pg` — persistance PostgreSQL en dev entre redémarrages Compose.

7.8 Bonnes pratiques appliquées

- images `alpine` ou `slim` lorsque possible ;
- pas de secrets dans les Dockerfiles — injection par `env` / secrets K8s ;
- builds reproductibles référencés en CI (job `docker`).

Chapitre 8

Kubernetes et Helm

8.1 Kubernetes et kind

8.1.1 Principe

Kubernetes orchestre des conteneurs sur un cluster : scheduling, réseau, stockage, auto-healing via sondes. **kind** (Kubernetes in Docker) instancie un cluster dont les nœuds sont des conteneurs Docker — idéal pour développement et CI sans cloud.

8.1.2 Configuration du cluster

`infrastructure/kubernetes/kind/cluster-config.yaml` définit :

- cluster nommé `pfa-stock` ;
- un nœud `control-plane` avec label `ingress-ready=true` ;
- mapping `hostPort` 80 et 443 vers le nœud (accès `http://localhost` depuis l'hôte).

Le kubeconfig généré est stocké dans `.kube/config` (gitignoré). Toute commande `kubectl` doit utiliser :

```
1 export KUBECONFIG="$(pwd)/.kube/config"
```

8.2 Namespaces

Terraform crée trois namespaces (`kubernetes.tf`) :

- `ingress-nginx` — contrôleur Ingress ;
- `monitoring` — kube-prometheus-stack ;
- `pfa-stock` — application métier.

Le namespace application n'est **pas** créé par le chart Helm (`create_namespace=false`) pour éviter les conflits « already exists » lors des réapplications.

8.3 Chart Helm pfa-stock

8.3.1 Métadonnées

`helm/pfa-stock/Chart.yaml` : chart version 1.0.0, application version 1.0.0.

8.3.2 Values principales

`values.yaml` paramètre images (`pfa-stock-api`, `pfa-stock-web`, tag `latest`), credentials PostgreSQL, Ingress host `localhost`, endpoint LocalStack in-cluster, activation du ServiceMonitor (`monitoring.enabled: true`).

TABLE 8.1 – Ressources du chart `pfa-stock`

Template	Kind	Détail
<code>postgres.yaml</code>	StatefulSet + Service	Volume 1Gi, image postgres :16-alpine
<code>api.yaml</code>	Deployment + Service	Env depuis Secret/ConfigMap, probes /health, ServiceMonitor
<code>web.yaml</code>	Deployment + Service	nginx, port 80
<code>localstack.yaml</code>	Deployment + Service	SERVICES=s3
<code>secret.yaml</code>	Secret	JWT, mot de passe PG, DATABASE_URL complète
<code>configmap.yaml</code>	ConfigMap	CORS, variables AWS
<code>ingress.yaml</code>	Ingress	class nginx, /api → api :8000, / → web :80

8.3.3 Ressources déployées

8.3.4 Ingress et routage

L’Ingress `pfa-stock-ingress` utilise `ingressClassName: nginx`. Contrairement à certaines configurations initiales, **aucune annotation de rewrite** ne supprime le préfixe `/api` — l’API attend les chemins complets `/api/v1/...`.

La figure 8.1 résume la répartition des chemins HTTP sur l’hôte `localhost` (port 80 exposé par ingress-nginx via `hostPort`).

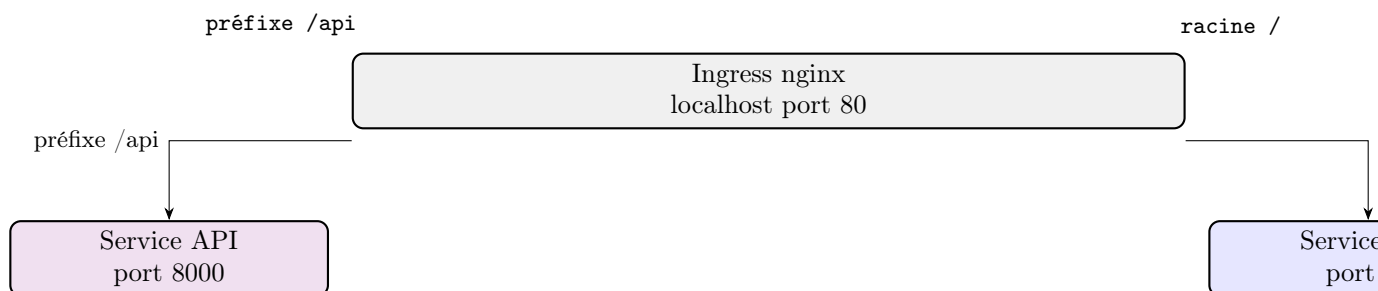


FIGURE 8.1 – Routage Ingress — chemins préservés vers l’API

Le frontend est construit avec `VITE_API_URL=http://localhost` : les requêtes XHR ciblent donc `http://localhost/api/v1/...`, qui atteignent le pod API sans proxy nginx intermédiaire dans le pod web (contrairement au mode Compose où nginx du conteneur `web` peut proxifier `/api/`).

8.3.5 Sondes API

- `startupProbe` — jusqu’à 30 échecs × 5 s pour les démarrages lents (PostgreSQL) ;
- `livenessProbe` — `/health` toutes les 10 s.

8.3.6 ServiceMonitor

Si `monitoring.enabled`, le template crée un `ServiceMonitor` CRD avec label `release: kube-prometheus-stack`, endpoint `/metrics` sur le port nommé `http`, intervalle 30 s.

8.4 ingress-nginx sur kind

Le fichier `infrastructure/terraform/values/ingress-nginx-kind.yaml` adapte le chart upstream :

- Service en ClusterIP avec `hostPort` 80/443 sur le nœud `ingress-ready` ;

- désactivation des webhooks d'admission (souvent problématiques sur kind) ;
- timeout Helm porté à 600 s lors du `helm_release` Terraform.

8.5 Cycle de déploiement manuel

Via `make cluster` (`scripts/demarrer-cluster.sh`) :

1. LocalStack up ;
2. `kind create` + load images ;
3. `terraform init` et `apply` dans le conteneur outils ;
4. application accessible sur port 80.

`scripts/reparder-cluster.sh` désinstalle le release applicatif et relance Terraform en cas d'état incohérent.

8.6 Limites du déploiement local

- un seul nœud — pas de tolérance aux pannes ;
- images `IfNotPresent` / `latest` — en production on épinglerait des digests ;
- secrets en clair dans values pour la démo — à externaliser en prod.

Chapitre 9

Infrastructure as Code — Terraform

9.1 Objectifs de Terraform dans StockFlow

Terraform décrit l'infrastructure de façon **déclarative** et **idempotente** : le même code peut être réappliqué pour converger vers l'état désiré. Dans ce projet, il :

1. provisionne un bucket S3 sur LocalStack ;
2. installe les charts Helm ingress-nginx, kube-prometheus-stack et pfa-stock ;
3. publie le dashboard Grafana via ConfigMap.

9.2 Providers

`providers.tf` configure trois providers :

9.2.1 AWS → LocalStack

Listing 9.1 – Provider AWS (extrait)

```
1 provider "aws" {
2   access_key = "test"
3   secret_key = "test"
4   s3_use_path_style = true
5   skip_credentials_validation = true
6   endpoints { s3 = var.localstack_endpoint }
7 }
```

Les flags `skip_*` désactivent les vérifications impossibles hors AWS réel. Le endpoint S3 pointe vers LocalStack (<http://localhost:4566> depuis le conteneur Terraform en réseau host, ou hostname adapté).

9.2.2 Kubernetes et Helm

Le provider Kubernetes et le provider Helm utilisent `config_path = "/root/.kube/config"` — fichier monté depuis `.kube/` du dépôt dans le conteneur `terraform` de `compose.ouils.yml`.

9.3 Variables

`variables.tf` expose :

- `localstack_endpoint`, `aws_region` ;
- `s3_bucket_name` (défaut `pfa-stock-exports`) ;
- `namespace_app`, `namespace_monitoring` ;
- `grafana_admin_password` (sensible, passé en `TF_VAR_` ou `.env`).

9.4 Ressources par fichier

9.4.1 kubernetes.tf

Crée les namespaces `monitoring`, `ingress-nginx`, `pfa-stock` avant les releases Helm — ordre garanti et pas de double création par le chart.

9.4.2 localstack.tf

- `aws_s3_bucket.exports`;
- `aws_s3_bucket_versioning` activé;
- output `s3_bucket_arn`.

9.4.3 helm.tf

Trois `helm_release` avec dépendances :

TABLE 9.1 – Releases Helm gérées par Terraform

Ressource	Chart	Version	Dépend de
<code>ingress_nginx</code>	<code>ingress-nginx</code>	4.11.3	<code>namespace</code>
<code>kube_prometheus</code>	<code>kube-prometheus-stack</code>	65.5.0	<code>ingress_nginx</code>
<code>pfa_stock</code>	<code>chart local</code>	—	<code>kube_prometheus</code>

`pfa_stock` monte le chart depuis `../helm/pfa-stock` et fixe `monitoring.releaseLabel` via `set`.

9.4.4 grafana_dashboard.tf

Crée une ConfigMap `grafana-dashboard-pfa-stock-api` dans le namespace `monitoring`, contenu = fichier JSON du dépôt, label `grafana_dashboard=1` pour le sidebar Grafana.

9.5 Exécution opérationnelle

```
1 docker compose -f docker/compose.ouutils.yml run --rm terraform init
2 docker compose -f docker/compose.ouutils.yml run --rm terraform apply
```

Le state est stocké localement (`terraform.tfstate`, gitignoré). Pas de backend distant — acceptable pour démo locale, à remplacer par S3+DynamoDB en équipe.

9.6 Validation en CI

Le job `terraform` exécute :

- `terraform init -backend=false`;
- `terraform fmt -check`;
- `terraform validate`.

Aucun `apply` en CI — pas de modification d'infra sur les runners GitHub.

9.7 Fichiers Terraform et responsabilités

L'ordre d'application respecte les `depends_on` : namespaces, puis `ingress`, puis `monitoring`, puis l'application — le `ServiceMonitor` de l'API nécessite un `Prometheus Operator` déjà présent.

TABLE 9.2 – Fichiers du répertoire `infrastructure/terraform/`

Fichier	Rôle
<code>providers.tf</code>	Providers AWS (LocalStack), Kubernetes, Helm
<code>variables.tf</code>	Mots de passe, bucket S3, kubeconfig
<code>namespaces.tf</code>	Namespaces créés avant les charts
<code>s3.tf</code>	Bucket d'export sur LocalStack
<code>helm.tf</code>	Releases ingress-nginx, kube-prometheus-stack, application
<code>grafana-dashboard.tf</code>	ConfigMap du dashboard JSON
<code>outputs.tf</code>	URLs et rappels <code>post-apply</code>

9.8 Migration vers un cloud réel

TABLE 9.3 – Adaptations pour production cloud

Composant local	Équivalent production
LocalStack S3	S3 AWS réel + IAM
kind	EKS, GKE ou AKS
Provider endpoints factices	Credentials IAM / IRSA
State local	Backend S3 chiffré + verrou DynamoDB

Le code Helm et applicatif restent largement inchangés ; seuls providers, backends et values diffèrent.

Chapitre 10

Observabilité — Prometheus et Grafana

10.1 Objectifs

L'observabilité vise à répondre, en exploitation :

- l'API reçoit-elle du trafic ? à quel rythme ?
- quelle est la latence (p50, p95, p99) ?
- y a-t-il des erreurs 5xx ?
- quelles routes sont les plus sollicitées ?

StockFlow adopte la stack **Prometheus** (collecte) + **Grafana** (visualisation), standard dans l'écosystème Kubernetes.

10.2 Instrumentation applicative

Le package `prometheus-fastapi-instrumentator` expose automatiquement sur `GET /metrics` :

- compteur `http_requests_total` (labels method, handler, status) ;
- histogrammes de durée de requête ;
- métriques en vol (`inprogress`).

Aucune modification manuelle par route n'est nécessaire pour les métriques HTTP de base.

10.3 kube-prometheus-stack

Terraform installe le chart communautaire `kube-prometheus-stack` (version 65.5.0) avec des values dans `monitoring/kube-prometheus-values.yaml` :

- **Grafana** activé, mot de passe admin via `set_sensitive` Terraform ;
- **sidecar dashboards** — surveille les ConfigMaps labellisées `grafana_dashboard: "1"` dans tous les namespaces ;
- **Prometheus** — rétention 2 jours, ressources limitées pour kind ;
- composants control-plane désactivés (scheduler, etcd, etc.) pour alléger le cluster de démo.

10.4 ServiceMonitor

Le CRD `ServiceMonitor` (Prometheus Operator) décrit comment scraper une cible. Le chart `pfa-stock` crée :

Listing 10.1 – Extrait ServiceMonitor (conceptuel)

```
1 spec:
2   selector:
3     matchLabels:
4       app: api
5   endpoints:
6     - port: http
7       path: /metrics
8       interval: 30s
```


Le label `release: kube-prometheus-stack` associe ce monitor à l'instance Prometheus déployée par le chart homonyme.

10.5 Flux de métriques

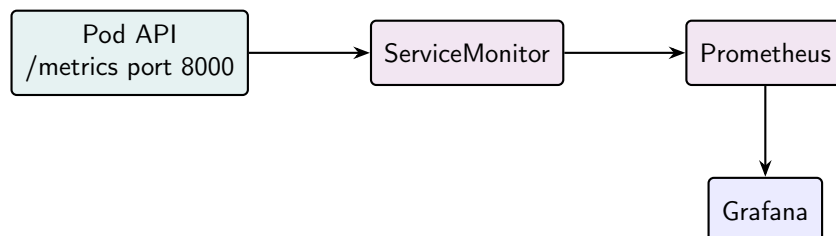


FIGURE 10.1 – Chemin des métriques

10.6 Dashboard Grafana

Le fichier `monitoring/dashboards/pfa-stock-api.json` définit le tableau de bord *PFA Stock — API FastAPI* avec panneaux :

- débit de requêtes ;
- erreurs 5xx ;
- latences p50 / p95 / p99 ;
- répartition par route et par code HTTP ;
- requêtes en cours.

Terraform injecte ce JSON en ConfigMap ; le sidecar Grafana le charge automatiquement — le dashboard est **versionné comme le code**.

10.7 Accès opérationnel

```
1 make grafana
2 # ou scripts/port-forward-grafana.sh
3 # http://localhost:3001      admin / GrafanaAdmin123!
```

Pendant une démonstration, naviguer dans l'application fait varier les courbes — preuve que le scrape est effectif.

10.8 Mode développement

En `make dev`, Prometheus/Grafana ne sont pas déployés par défaut. Les métriques restent accessibles sur `http://localhost:8000/metrics` pour `curl` ou un Prometheus local ad hoc.

10.9 Alerting

Alertmanager est inclus dans kube-prometheus-stack mais les règles d'alerte custom (stock critique, API down prolongé) ne sont pas configurées dans le périmètre actuel — extension naturelle pour une phase ultérieure.

Chapitre 11

Intégration continue — GitHub Actions

11.1 Principes

La CI (*Continuous Integration*) valide chaque modification avant intégration sur `main` ou `develop`. Le workflow `.github/workflows/ci.yml` lance **six jobs indépendants** sur des runners Ubuntu — feedback parallèle, périmètres isolés. Aucun secret cloud n'est requis : les jobs s'appuient uniquement sur les actions officielles GitHub, Docker et Hashicorp.

11.2 Déclencheurs

- `push` sur branches `main` et `develop` ;
- `pull_request` ciblant `main`.

Aucun déploiement automatique (pas de CD) : les images buildées ne sont pas poussées vers un registry ; Terraform n'est pas appliqué sur GitHub.

11.3 Vue d'ensemble

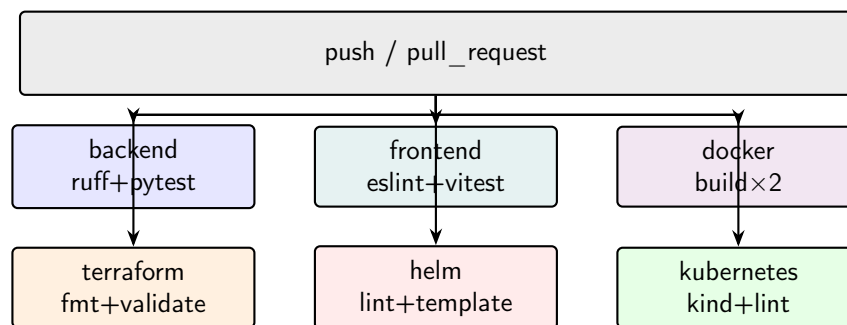


FIGURE 11.1 – Pipeline CI — six jobs parallèles

11.4 Structure du fichier workflow

Le fichier source fait 91 lignes : en-tête `on:` (déclencheurs), puis six blocs `jobs:` sans dépendances `needs:` entre eux. L'extrait suivant montre la déclaration des déclencheurs et du premier job :

Listing 11.1 – Extrait `.github/workflows/ci.yml`

```
1 name: CI PFA Stock
2 on:
3   push:
4     branches: [main, develop]
5   pull_request:
6     branches: [main]
7 jobs:
```

```

8  backend:
9    runs-on: ubuntu-latest
10   services:
11     postgres:
12       image: postgres:16-alpine
13       ...
14   steps:
15     - uses: actions/checkout@v4
16     - uses: actions/setup-python@v5
17     - run: ruff check app tests
18     - run: pytest -q

```

Cette structure « fan-out » minimise le temps de feedback : le chemin critique est celui du job le plus lent (souvent docker ou kubernetes), pas la somme de tous les jobs.

11.5 Job backend

1. Service container `postgres:16-alpine` avec healthcheck (`pg_isready`) sur le port 5432;
2. Python 3.12, `pip install -r requirements.txt`;
3. `ruff check app tests` — lint statique sans exécuter le code;
4. `pytest -q` avec `DATABASE_URL=postgresql://stock:stock@localhost:5432/stock_db` injectée par le workflow.

Note importante : les tests actuels passent par `tests/conftest.py`, qui force SQLite en mémoire lorsque `MODE_TEST=1` (fixture de session). La variable `DATABASE_URL` Postgres en CI est donc *prête* pour de futurs tests d'intégration, mais n'est pas encore le moteur effectif des assertions `pytest`. Cette distinction évite de croire que la CI exécute déjà une suite Postgres complète.

11.6 Job frontend

Node 20, `npm install`, `npm run lint`, `npm test - --run`. Valide le typage implicite via build de tests et les règles ESLint React.

11.7 Job docker

```

docker/setup-buildx-action puis :
— docker build -t pfa-stock-api:latest backend;
— docker build -t pfa-stock-web:latest -build-arg VITE_API_URL=http://localhost:8000 frontend.

```

Garantit que les Dockerfiles restent valides sur une machine neuve.

11.8 Job terraform

Dans `infrastructure/terraform` : `init -backend=false`, `fmt -check`, `validate`. Détecte erreurs de syntaxe et formatting non standard.

11.9 Job helm

`helm lint helm/pfa-stock` et `helm template ... > /dev/null` — rendu des manifests sans déploiement.

11.10 Job kubernetes

`helm/kind-action` crée un cluster `pfa-ci` avec la même config `kind` que le projet, puis `helm lint` à nouveau. Vérifie la compatibilité avec un API server Kubernetes réel ; **ne déploie pas** le chart (`helm install absent`).

11.11 Comparaison avec `make test`

TABLE 11.1 – CI vs commandes locales

Vérification	CI	<code>make test</code> / <code>make lint</code>
<code>pytest</code>	Oui	Oui (conteneur api)
<code>vitest</code>	Oui	Oui (conteneur node)
<code>ruff</code>	Oui	<code>make lint</code>
<code>eslint</code>	Oui	Non (Makefile)
<code>docker build</code>	Oui	Non
<code>terraform / helm</code>	Oui	Non

11.12 Matrice des actions tierces

TABLE 11.2 – Actions GitHub utilisées par job

Job	Action	Rôle
backend	<code>actions/setup-python@v5</code>	Toolchain Python 3.12
frontend	<code>actions/setup-node@v4</code>	Node 20
docker	<code>docker/setup-buildx-action@v3</code>	BuildKit pour builds reproductibles
terraform	<code>hashicorp/setup-terraform@v3</code>	Binaire Terraform
helm	<code>azure/setup-helm@v4</code>	CLI Helm
kubernetes	<code>helm/kind-action@v1</code>	Cluster éphémère <code>kind</code>
tous	<code>actions/checkout@v4</code>	Clone du dépôt

11.13 Échecs et gouvernance

Si un job échoue, le workflow est rouge. Les autres jobs terminent leur exécution (pas d'`fail-fast` global). Sur une organisation GitHub stricte, les PR peuvent exiger tous les checks verts avant merge. En local, `make lint` et `make test` reproduisent partiellement le périmètre applicatif mais pas la validation infrastructurelle.

11.14 Évolutions CD possibles

1. Push des images vers GHCR avec tag `${github.sha}` ;
2. `helm upgrade` sur un cluster staging avec ces tags ;
3. `terraform plan` commenté sur les PR ;
4. scan Trivy des images ;
5. tests e2e Playwright post-déploiement éphémère.

Chapitre 12

Sécurité, tests et qualité

12.1 Modèle de menace (périmètre local)

Le déploiement documenté cible un **environnement local de démonstration**. Les menaces principales sont : accès non autorisé à l'API, fuite de secrets dans Git, injection SQL (mitigée par ORM), et configuration CORS trop permissive.

12.2 Authentification et autorisation

- Mots de passe hashés bcrypt — pas de stockage clair ;
- JWT signé HS256 — secret `SECRET_JWT` injecté par variable d'environnement ou Secret K8s ;
- Durée de vie configurable (`duree_token_minutes`) ;
- Contrôles de rôle sur DELETE et export CSV (vérification `role == admin`).

12.3 Secrets et configuration

TABLE 12.1 – Gestion des secrets

Secret	Stockage
<code>SECRET_JWT</code>	Secret K8s / <code>.env</code> (gitignoré)
<code>DATABASE_URL</code>	Secret K8s (mot de passe PG inclus)
Mots de passe démo	Seed — à changer en prod
Grafana admin	<code>TF_VAR_grafana_admin_password</code>

`.env.example` documente les clés sans valeurs sensibles réelles. `.gitignore` exclut `.env`, `.kube/`, `*.tfstate`.

12.4 CORS

`CORS_ORIGINS` liste les origines autorisées (frontend dev sur `:3000`, cluster sur `http://localhost`). En production, la liste serait restreinte au domaine client réel.

12.5 Réseau Kubernetes

Les Services clusterIP ne sont pas exposés sauf via Ingress (80/443). LocalStack et PostgreSQL ne sont pas publiés sur l'hôte en mode cluster (sauf port-forward manuel pour debug).

12.6 Tests automatisés

12.6.1 Backend (pytest)

Quatre tests couvrent :

- connexion + liste produits ;
- création produit ;
- entrée de stock ;
- sortie refusée si stock insuffisant (`stock_insuffisant`).

Fixtures : client FastAPI, jeton admin, base SQLite mémoire, override de `obtenir_session`.

Le fichier `conftest.py` isole chaque test :

Listing 12.1 – Stratégie de test (extrait conftest)

```

1 os.environ["MODE_TEST"] = "1"
2 # SQLite :memory: + create_all avant chaque fonction de test
3 @pytest.fixture
4 def client():
5     with TestClient(application) as c:
6         yield c

```

Cette approche garantit un état vierge par test sans dépendre d'un Postgres déjà peuplé par le seed de production.

12.6.2 Frontend (Vitest)

Test de rendu de la page connexion avec providers Auth/Toast et MemoryRouter.

12.6.3 Smoke tests (make verify)

`scripts/verifier.sh` enchaîne :

- mode dev ou cluster ;
- curl sur `/health`, login, liste produits, page web ;
- en cluster : vérification pods Running.

12.7 Analyse statique

- **ruff** — backend Python (CI + `make lint`) ;
- **eslint** — frontend TypeScript/React (CI).

12.8 Pistes de durcissement production

- TLS terminé par cert-manager + Let's Encrypt ;
- rotation des secrets via External Secrets Operator ;
- politiques NetworkPolicy limitant le trafic inter-pods ;
- rate limiting sur l'Ingress ;
- audit des dépendances (`pip audit`, `npm audit`).

Chapitre 13

Exploitation et scripts d'automatisation

13.1 Makefile comme point d'entrée

Le Makefile normalise les opérations courantes (tableau 13.1).

TABLE 13.1 – Commandes Make principales

Cible	Action
prerequis	Vérifie Docker et Compose
dev	Lance Compose dev + init S3
cluster	Bootstrap kind + Terraform
demo / demo-cluster	Démarrage + tests + liens
test	pytest + vitest dans Docker
verify	Smoke tests (MODE=dev cluster)
links	Affiche URLs et identifiants
grafana	Port-forward Grafana :3001
destroy	Arrêt complet
lint	ruff + eslint

13.2 Scripts shell

TABLE 13.2 – Scripts du répertoire `scripts/`

Script	Rôle
demarrer-dev.sh	Compose dev, bucket S3
demarrer-cluster.sh	LocalStack, kind, build/load images, terraform apply
demarrer-tout.sh	Orchestration demo dev ou cluster + verify + links
demarrer-grafana.sh	Port-forward Grafana en arrière-plan
init-localstack.sh	Création bucket via aws-cli dans conteneur
kind-docker.sh	Exécute binaire kind via image golang + socket Docker
verifier.sh	Tests smoke HTTP et état pods
afficher-acces.sh	Imprime credentials et URLs
arreter-tout.sh	Down compose, delete kind, stop port-forwards
reparer-cluster.sh	Helm uninstall app + terraform re-apply

13.3 Prérequis machine hôte

Docker avec Compose V2, Make, curl (pour verify). **Pas** de Python, Node, Terraform, kubectl, Helm, kind installés nativement — réduisant les dérives de versions.

13.4 Identifiants et URLs

13.4.1 Mode dev

- UI : `http://localhost:3000`
- API : `http://localhost:8000`, docs /docs
- Admin : `admin@stock.tn` / `Admin123!`

13.4.2 Mode cluster

- Application : `http://localhost`
- Grafana : `http://localhost:3001` après `make grafana`
- Grafana : `admin` / `GrafanaAdmin123!`

13.5 Dépannage courant

Ingress 404 sur /api Vérifier absence de `rewrite incorrect` ; chart actuel préserve `/api`.

Pod API CrashLoop Vérifier Secret `DATABASE_URL` — historiquement causé par expansion de variable avant définition du secret.

Namespace déjà existant Namespace créé par Terraform ET chart — corrigé en supprimant `namespace.yaml` du chart.

kubectl connection refused Exporter `KUBECONFIG` vers `.kube/config`.

Helm timeout ingress Values kind avec `hostPort` et `webhooks` désactivés, `timeout 600s`.

13.6 Logs et diagnostic

```
1 kubectl logs -n pfa-stock deployment/api -f
2 kubectl describe pod -n pfa-stock -l app=api
3 docker compose -f docker/compose.dev.yml logs api
```

13.7 Sauvegarde

En cluster, les données PostgreSQL résident sur un PVC 1 Gi. Une stratégie de backup production utiliserait `pg_dump` planifié ou snapshots de volume cloud. Les exports CSV S3 sur LocalStack ne survivent pas à `make destroy` sans volume persistant LocalStack — acceptable pour démo.

Chapitre 14

Conclusion et perspectives

14.1 Bilan

Le projet **StockFlow** démontre qu’une application métier de gestion de stock peut être livrée conjointement avec une chaîne DevOps complète : conteneurisation, orchestration Kubernetes locale, infrastructure as code, observabilité et intégration continue — tout en restant reproductible via Docker sur une workstation standard.

Les choix techniques (FastAPI, React, PostgreSQL, kind, Helm, Terraform, Prometheus/Grafana, GitHub Actions) forment un écosystème cohérent et largement documenté dans l’industrie, facilitant la montée en compétence et la migration ultérieure vers un cloud public.

14.2 Objectifs atteints

- Fonctionnalités métier couvrant catalogue, mouvements, alertes, tableau de bord et export ;
- Deux modes d’exécution (Compose et cluster) partageant les mêmes images ;
- Pipeline CI à six voies validant code, images et manifests ;
- Dashboard Grafana alimenté par métriques réelles de l’API ;
- Automatisation `make demo-cluster` pour démonstration bout en bout.

14.3 Limites connues

- Cluster mono-nœud, pas de HA ;
- Secrets et mots de passe de démo non durcis ;
- CI ne déploie pas ni ne pousse les images ;
- Tests d’intégration Postgres en CI non exploités (SQLite en tests unitaires) ;
- Pas de TLS ni de gestion centralisée des secrets (Vault, etc.).

14.4 Perspectives

1. **Cloud managé** — EKS/GKE, RDS, S3 réel, backend Terraform distant ;
2. **CD** — publication d’images et déploiement staging automatique après CI verte ;
3. **GitOps** — Flux ou Argo CD pour reconcilier l’état cluster sur Git ;
4. **Sécurité** — TLS, NetworkPolicies, scan d’images, rotation JWT ;
5. **Fonctionnel** — réapprovisionnement automatique, notifications alertes, audit trail enrichi ;
6. **Tests** — couverture e2e, tests de charge (k6), intégration Postgres en CI.

14.5 Mot de fin

StockFlow illustre une approche où le code applicatif et l’infrastructure évoluent dans le même dépôt, revus par la même pipeline CI. Cette unification réduit l’écart entre « ça marche en local » et « ça tourne comme en production », condition préalable à tout déploiement sérieux.

Annexe A

Annexe A — Référence API REST

Préfixe commun : `/api/v1`. Authentification : `Authorization: Bearer <jeton>` sauf connexion.

A.1 Auth

Méthode	Chemin	Corps / réponse
POST	<code>/auth/connexion</code>	JSON <code>{email, mot_de_passe}</code> → <code>{jeton_acces}</code>
GET	<code>/auth/moi</code>	Profil utilisateur courant

A.2 Catégories

CRUD standard : `GET/POST /categories`, `GET/PUT/DELETE /categories/{id}`. `DELETE` réservé admin.

A.3 Produits

`GET /produits`, `GET /produits/alertes`, `POST /produits`, `GET/PUT/DELETE /produits/{id}`.
Champs création : `nom`, `reference_sku`, `quantite`, `seuil_alerte`, `id_categorie`.

A.4 Mouvements

`GET /mouvements`, `POST /mouvements` avec `type_mouvement` (`entree|sortie`), `quantite`, `id_produit`, motif optionnel.
Erreur métier : 400 `stock_insuffisant`.

A.5 Tableau de bord

`GET /tableau-de-bord` — JSON avec compteurs et liste `produits_alerte`.

A.6 Exports

`POST /exports/produits-csv` — admin uniquement, retourne informations sur l'objet S3 créé.

A.7 Santé et métriques

`GET /health` — `{"statut":"ok"}` (ou équivalent).
`GET /metrics` — format Prometheus texte.

A.8 Exemples de payloads JSON

A.8.1 Connexion

Requête :

```
1 POST /api/v1/auth/connexion
2 {"email": "admin@stock.tn", "mot_de_passe": "Admin123!"}
```

Réponse 200 :

```
1 {"jeton_acces": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."}
```

A.8.2 Création de produit

```
1 POST /api/v1/produits
2 {
3   "nom": "Clavier m canique",
4   "reference_sku": "KB-MECH-01",
5   "quantite": 25,
6   "seuil_alerte": 5,
7   "id_categorie": 1
8 }
```

A.8.3 Mouvement de sortie

```
1 POST /api/v1/mouvements
2 {
3   "type_mouvement": "sortie",
4   "quantite": 3,
5   "id_produit": 2,
6   "motif": "Vente comptoir"
7 }
```

A.8.4 Tableau de bord (extrait de réponse)

```
1 {
2   "nombre_produits": 12,
3   "nombre_categories": 4,
4   "nombre_mouvements": 28,
5   "nombre_alertes": 2,
6   "produits_alerte": [ ... ]
7 }
```

A.9 Codes d'erreur HTTP courants

A.10 Exemples cURL (mode Compose)

Avec l'API sur localhost:8000 et un jeton dans la variable \$TOKEN :

```
1 curl -s http://localhost:8000/health
2 curl -s -H "Authorization: Bearer $TOKEN" \
3   http://localhost:8000/api/v1/tableau-de-bord | jq
4 curl -s -X POST -H "Authorization: Bearer $TOKEN" \
```

TABLE A.1 – Erreurs API typiques

Code	detail	Contexte
401	jeton_invalide / non authentifié	Jeton absent, expiré ou signature incorrecte
403	accès refusé	Rôle insuffisant (ex. export admin)
404	produit_introuvable	ID produit inexistant
400	stock_insuffisant	Sortie dépassant le stock disponible
422	validation Pydantic	Corps JSON mal formé ou champs manquants

```
5 -H "Content-Type: application/json" \  
6 -d '{"type_mouvement":"entree","quantite":10,"id_produit":1}' \  
7 http://localhost:8000/api/v1/mouvements
```

En mode cluster, remplacer l'hôte par `http://localhost/api/v1/...` (préfixe Ingress).

Annexe B

Annexe B — Configuration et versions

B.1 Variables d’environnement (`.env.example`)

TABLE B.1 – Variables d’environnement principales

Variable	Description
DATABASE_URL	URL SQLAlchemy PostgreSQL
SECRET_JWT	Clé signature JWT
CORS_ORIGINS	Liste séparée par virgules
VITE_API_URL	URL API pour build frontend
AWS_ENDPOINT_URL	LocalStack ou AWS
AWS_ACCESS_KEY_ID / SECRET	Credentials S3
S3_BUCKET_EXPORTS	Nom du bucket
POSTGRES_*	Paramètres PostgreSQL (Compose/Helm)

B.2 Versions d’outils (référence)

Composant	Version
Python	3.12
Node.js	20
PostgreSQL	16
Terraform	1.9 (image outils)
Helm chart ingress-nginx	4.11.3
kube-prometheus-stack	65.5.0
LocalStack	3.x

B.3 Fichiers de référence rapide

- Workflow CI : `.github/workflows/ci.yml`
- Compose dev : `docker/compose.dev.yml`
- Chart : `helm/pfa-stock/values.yml`
- Terraform : `infrastructure/terraform/`
- Dashboard : `monitoring/dashboards/pfa-stock-api.json`